

QUIC-Native Load Balancer

Proof of Concept

Fire-and-Forget Load Balancing via QUIC Server-Side Migration

PoC Documentation • July 2026

This document describes the design and implementation of a QUIC-native load balancer that leverages server-side connection migration (RFC 9000, Section 9.6) to achieve "fire-and-forget" load balancing. Unlike traditional load balancers (HAProxy, NGINX, Maglev) that remain in the data path for the entire connection lifetime, our approach handles only the initial handshake (1 RTT), selects a backend, and then steps out of the data path entirely. After migration, the client communicates directly with the backend server with zero frontend overhead.

1. Overview

Traditional load balancers sit in the data path for the **entire connection lifetime**. Every packet between client and server must traverse the LB, making it a throughput bottleneck and a single point of failure. This is true for both L4 (Maglev, LVS) and L7 (NGINX, Envoy, HAProxy) load balancers.

QUIC's server-side migration mechanism (the `preferred_address` transport parameter) offers a fundamentally different approach:

- The frontend handles **only the TLS handshake** (1 RTT), during which it selects a backend and advertises the backend's address as the preferred address.
- After the handshake, the client migrates directly to the backend. The frontend exports the cryptographic state and sends it to the backend.
- Once migration completes, the frontend has **zero per-connection state** and processes zero data packets for that connection.
- This is "fire-and-forget" load balancing: the frontend dispatches and the backend serves directly.

Key insight: The frontend's work is bounded by $O(1)$ per connection (handshake only), not $O(n)$ per packet. This eliminates the LB as a bandwidth bottleneck.

2. Architecture

The PoC runs on a four-machine testbed, all on the same LAN (141.217.168.x):

Role	Machine	IP Address	Binary
Frontend (LB)	opti7040	141.217.168.152	lb-frontend
Backend 1	homeserver2	141.217.168.143	preferred-server
Backend 2	Proxmox VM	141.217.168.200	preferred-server
Client	optiplex7010	141.217.168.127	Firefox / neqo-client

Connection Flow

1. Client connects to **frontend** (141.217.168.152:4433) via QUIC.
2. Frontend completes the TLS 1.3 handshake with the client (1 RTT).
3. Frontend selects a backend based on the configured LB policy (round robin or random).
4. Frontend includes the selected backend's address as `preferred_address` in the TLS handshake.
5. Frontend exports ~445 bytes of migration state (TLS secrets, CIDs, packet numbers).
6. Migration state is sent via TCP to the selected backend's port 9999.
7. Client sends `PATH_CHALLENGE` to the backend's address.
8. Backend imports the crypto state, decrypts `PATH_CHALLENGE`, sends `PATH_RESPONSE`.
9. Client validates `PATH_RESPONSE` and migrates. All subsequent data flows directly client <-> backend.
10. Frontend tears down the connection and is ready for the next one.

3. Load Balancing Policies

The frontend supports multiple load balancing policies, selected via the `LB_POLICY` environment variable.

Round Robin (default)

Cycles through backends in order: `[0, 1, 0, 1, ...]`. This provides perfectly even distribution across backends and is deterministic. It is the default policy when no `LB_POLICY` environment variable is set.

Random

Pseudo-random selection per connection. Each incoming connection is assigned to a backend chosen uniformly at random. Over many connections, this converges to even distribution but may have short-term imbalances. Selected with `LB_POLICY=random`.

Policy	Selection	Distribution	Use Case
Round Robin	Sequential cycling	Perfectly even	Default; homogeneous backends
Random	Pseudo-random per conn	Statistically even	Simple; no state needed

4. Implementation Details

Binary: lb-frontend

The load balancer frontend is implemented as the `lb-frontend` binary in the `migration-test` crate. It is a purpose-built binary separate from the standard `primary-server`.

Per-Connection Lifecycle

- Creates a **new `Http3Server` instance per connection** with the selected backend's address configured as the `preferred_address` transport parameter.
- After the TLS handshake completes and migration state is exported, the server instance is **torn down**. No per-connection state persists at the frontend.
- The frontend then loops back to wait for the next incoming connection.

State Transfer

Migration state (~445 bytes) is sent via a direct TCP connection from the frontend to the selected backend's port 9999. This is a one-shot send with no response expected. The state includes TLS secrets, connection IDs, client address, QUIC version, and packet numbers.

Per-Connection Stats

The frontend prints per-connection statistics including the selected backend, handshake duration, and state export time. It also maintains and prints a running backend distribution table showing how many connections have been sent to each backend.

5. How to Run

Start the components in the following order:

Step 1: Start Backend 1 (homeserver2, .143)

```
preferred-server 141.217.168.143:4433 9999
```

Step 2: Start Backend 2 (Proxmox VM, .200)

```
preferred-server 141.217.168.200:4433 9999
```

Step 3: Start Frontend (opti7040, .152)

```
lb-frontend 0.0.0.0:4433 141.217.168.143:4433 141.217.168.200:4433
```

Step 4: Connect from Client (.127)

Open Firefox and navigate to `https://141.217.168.152:4433/`. The first load triggers the TCP bootstrap (Alt-Svc). On refresh, Firefox uses HTTP/3 and the migration-based load balancing takes effect.

To select a policy, set the environment variable before starting the frontend:

```
LB_POLICY=random lb-frontend 0.0.0.0:4433 141.217.168.143:4433 141.217.168.200:4433
```

6. Advantages Over Traditional Load Balancers

The following table compares the QUIC migration-based load balancer against two established approaches: L4 pass-through (Maglev) and L7 reverse proxy (NGINX/Envoy).

Feature	Maglev (L4)	NGINX/Envoy (L7)	QUIC Migration LB
LB in data path	Always	Always	Handshake only
TLS termination at LB	No (pass-through)	Yes	Handshake only
Sees plaintext	No	Yes	No (after migration)
Per-conn state at LB	Flow table	Session	None after migration
Bandwidth bottleneck	LB throughput	LB throughput	None (direct path)

The QUIC migration LB combines the privacy benefits of L4 (no plaintext visibility) with the routing intelligence of L7 (backend selection at handshake time), while eliminating the persistent bandwidth bottleneck of both approaches.

7. Limitations

One-shot migration: Once a connection migrates to a backend, it cannot be re-balanced. If the backend becomes overloaded after migration, the LB cannot intervene. This is inherent to the preferred_address mechanism.

Client support required: The client must support the preferred_address transport parameter. Firefox supports this; Chrome/Chromium does not (as of mid-2026). This limits real-world applicability until broader adoption.

Per-connection server instantiation: The current PoC creates a new Http3Server instance per connection. This is a PoC limitation, not a fundamental architectural constraint. A production implementation would reuse a single server instance with per-connection preferred_address selection.

8. Test Plan

The following tests validate correctness, distribution, and performance of the load balancer PoC.

Test 1: Distribution Verification

Send 10 sequential connections to the frontend and verify even distribution across backends. With round-robin policy, expect exactly 5 connections per backend. With random policy, expect approximately even distribution (statistical test).

Test 2: Handshake + Migration Latency

Measure per-connection latency from initial ClientHello to completed migration (PATH_RESPONSE validated by client). Break down into: handshake time (frontend), state transfer time (TCP push), and path validation time (PATH_CHALLENGE/RESPONSE round trip to backend). Target: total migration latency under 2 RTT.

Test 3: Baseline Comparison

Compare with HAProxy in TCP mode (L4 pass-through) as the baseline. Metrics: connection setup latency, sustained throughput (the QUIC LB should show zero throughput overhead after migration), and frontend CPU utilization under load.

Test	Method	Success Criteria
Distribution	10 sequential connections	5/5 split (round robin); ~even (random)
Latency	Per-connection timing	Total migration < 2 RTT on LAN
Baseline	HAProxy TCP mode comparison	Zero throughput overhead after migration